

device-side tuf trust enforcement (1.0.1-staged-rollout)

field	value
Revision	1
Created	2026-06-08
Last modified	2026-06-08
Status	planned (outline — depth follows 1.0.0-MVP; gated on the ADR-0002 §4.3 on-device-client spike) Normative outline for phase-1.0.1 device-side TUF (The Update Framework) trust enforcement. Layers a TUF metadata refresh/verify flow IN FRONT OF the existing MVP verify-before-apply gate (security/signing_verification.md §6) without re-signing or re-packaging artifacts. Server-side metadata is published by go-tuf/v2 (ADR-0002 §3.2); device-side enforcement becomes mandatory only after the ADR-0002 §4.3 step-2 client spike resolves the still-OPEN on-device client path (gomobile-wrapped go-tuf/v2 vs hand-rolled Kotlin). Adds the attack-class mitigations (rollback / freeze / mix-and-match / mirror-denial / key-compromise recovery) that MVP plain ED25519+SHA-256+AVB leaves open (ADR-0002 §1, §3.1). Composes with — does not replace — the ota-artifact-validator S2→S3 pipeline and the four native gates. The on-device TUF client (gomobile/JNI vs hand-rolled Kotlin vs an existing JVM client) is the dominant adoption cost and is UNVERIFIED/OPEN — no verified production-grade Kotlin/JVM TUF client is known to exist (ADR-0002 §8 item 1). go-tuf/v2 metadata/updater + metadata/trustedmetadata client-flow API signatures are UNVERIFIED
Status summary	
Issues	

field	value
Issues summary	(ADR-0002 §8 item 2). The security / Security-KMP catalogue bricks' fit for TUF-role / threshold / offline-key primitives is UNVERIFIED (ADR-0002 §8 item 9). Android-15 / RK3588 board specifics the client integrates against are UNVERIFIED (ADR-0002 §8 item 8). TUF metadata storage tables (deferred out of rollout_engine.md §11) are first specified here (§7) but their DDL lands in migration 002_*/003_* per the data-model owner, not here. Device client path is the long pole and OPEN; library, catalogue-brick, and board surfaces carried as UNVERIFIED; not made mandatory until the spike closes them.
Fixed	Rev 1: first device-side TUF spec for 1.0.1; pins the role model, the device verify-flow extension of the MVP gate, the bootstrap/rotation outline, the attack-class delta over plain signing, the go-tuf/v2(server) [?] device-client split with the two ADR-0002 spike options, and the composition with the ota-artifact-validator S-stage pipeline.
Fixed summary	Captured the 1.0.1 device-trust design as an outline traceable to ADR-0002 and the MVP signing spec, with every UNVERIFIED item carried, not invented.
Continuation	Run the ADR-0002 §4.3 step-1 (server publish path + Go reference refresh client end-to-end) and step-2 (on-device client ADR + spike) before any field of §6.3 is treated as decided; confirm the go-tuf/v2 updater/trustedmetadata client API against pkg.go.dev; confirm Security-KMP exposes (or can host without bespoke crypto) the TUF role/threshold/expiry primitives; define the offline key-custody ceremony (ADR-0002 §4.3 step-3) before mandating device enforcement; land the TUF metadata storage DDL (§7) in the migration owned by

field	value
	migration_002_design.md's successor.

table of contents

- [§1. purpose and scope](#)
- [§2. why tuf over plain signing \(the 1.0.1 delta\)](#)
- [§3. tuf metadata roles](#)
- [§4. attack classes tuf adds over plain signing](#)
- [§5. server-side \[?\] device-side split](#)
- [§6. device-side verification flow \(extends the verify-before-apply gate\)](#)
- [§7. key rotation + root-of-trust bootstrap on the device](#)
- [§8. composition with the ota-artifact-validator pipeline + native gates](#)
- [§9. data-model deltas \(tuf metadata storage\)](#)
- [§10. testing \(four-layer + attack-class\)](#)
- [§11. open / unverified items](#)
- [§12. compliance notes \(helixconstitution\)](#)
- [§13. sources](#)

The table-of-contents requirement is mandated by HelixConstitution §11.4.61 (UNVERIFIED clause number, per corpus convention). This document carries its ToC immediately after the metadata table.

§1. purpose and scope

This document specifies, for phase 1.0.1, how Helix OTA establishes **device-side TUF trust enforcement**: a metadata-driven refresh/verify flow that runs on the Android agent **before** the existing MVP verify-before-apply gate, so the device accepts an artifact only if a chain of TUF metadata (root → timestamp → snapshot → targets) proves the artifact is the **authorized, fresh, correct** target for that device — not merely a validly-signed blob.

It is the device half of ADR-0002's phased trust decision: "*MVP (1.0.0) ships plain per-artifact trust (signed artifact + SHA-256 + AVB). TUF (go-tuf/v2) is adopted server-side as the metadata/trust layer in 1.0.1+, with mandatory device-side enforcement gated behind an Android-client spike.*" (ADR-0002 §4). The server-side publish path is owned by ADR-0002 §3.2 and §4.3 step-1; this spec owns the **device** path and the seam where it joins the existing gate (security/signing_verification.md §6, §8).

In scope: the TUF role model as the device verifies it (§3); the attack-class delta over MVP plain signing (§2, §4); the server[?]device split and the two ADR-0002 client options (§5); the device verification flow extending the verify-before-apply gate (§6); device root-of-trust bootstrap and key rotation (§7); composition with the ota-artifact-validator S2→S3 pipeline and the four native gates (§8); the TUF metadata storage delta deferred out of rollout_engine.md §11 (§9); four-layer + attack-class testing (§10).

Out of scope / not re-decided here. TUF/Uptane *adoption* (ADR-0002, already decided phased). The *delivery* engine (native A/B `update_engine`; ADR-0001) — TUF is a trust layer, not a delivery engine (ADR-0002 §1). The MVP plain-signing scheme (`security/signing_verification.md`) — this spec **layers over** it byte-identically, never replaces it. Transport (ADR-0004). The staged-rollout engine and cohort assignment (`rollout_engine.md`) — referenced here only as the per-device decision seam a future Director consumes (§5, §7). Key custody *ceremony* mechanics (`key_management.md`; ADR-0002 §4.3 step-3) — referenced, not specified.

LOCKED context honored (not re-decided): native Android A/B (`update_engine` + AVB/dm-verity + auto-rollback) on device + a custom Go (Gin) control plane; Go + go-tuf/v2 server-side; Android-15-first (Orange Pi 5 Max / RK3588); the device-side client is the dominant, OPEN, UNVERIFIED cost (ADR-0002 §1, §4, §8).

§2. why tuf over plain signing (the 1.0.1 delta)

The MVP gate proves an artifact is **authentic** (signed by a trusted `key_id`) and **intact** (matches its declared SHA-256). It does **not** prove the artifact is the one this device is **authorized** to install *right now*. Plain per-artifact signature “*does not cover freshness, version monotonicity, or per-device targeting*” (`security/signing_verification.md` §3) and the whole MVP trust model “*collapses to a single signing key*” (ADR-0002 §3.1). The accepted MVP residual is explicit: “*no rollback/freeze/mix-and-match/mirror-denial mitigation until TUF in 1.0.1+*” (`security/signing_verification.md` §11 item 4; ADR-0002 §5.2).

TUF closes that delta with **separately-keyed, separately-expiring metadata roles** layered over the same byte-identical artifacts (ADR-0002 §3.2 additive/non-invasive layering). The device no longer trusts “a signed blob arrived”; it trusts “a current, threshold-signed, monotonic metadata chain says THIS digest is the authorized target.” A single popped signing key no longer lets an attacker ship arbitrary, stale, or device-targeted firmware — the attacker must also defeat threshold-held, offline-keyed roles (ADR-0002 §2 compromise-resilience driver). The per-attack-class breakdown is §4.

§3. tuf metadata roles

TUF defines four metadata roles (ADR-0002 §3.2; `research/stacks/tuf-go-tuf.md` §3). The device verifies them in a fixed order each refresh — **root** → **timestamp** → **snapshot** → **targets** (ADR-0002 §4.2 role order; `security/signing_verification.md` §8 seam 3). Each role is a signed JSON document with an `expires` field and a monotonic version; the device enforces both.

role	signs / asserts	key posture	device check (per refresh)	UNVERIFIED
root	The root of trust: which public keys + signature thresholds are	Offline , threshold-held; rotated via a chained	Verify the new root is signed by a threshold of the currently-trusted root	Offline-key custody ceremony is ADR-0002 §4.3 step-3,

role	signs / asserts	key posture	device check (per refresh)	UNVERIFIED
timestamp	authoritative for every role (including root itself). The key-distribution document.	root→root signature (§7).	keys (chain-of-trust walk from the device's pinned root, §7); enforce version strictly increasing; enforce not-expired.	UNVERIFIED until defined.
	Freshness beacon: a short-lived signature over the current snapshot metadata's hash+version. The “is the repo state current?” anchor.	Online (re-signed frequently by automation; ADR-0002 §4.3 step-3 timestamp re-signing automation).	Verify signed by a threshold of timestamp keys (per current root); enforce not-expired (tight expiry); it names the expected snapshot version+hash.	Re-signing cadence/ automation UNVERIFIED until ADR-0002 §4.3 step-3.
	Consistency: the version number (and, with consistent snapshots, hash) of the current targets metadata (and any delegated targets). Prevents mix-and-match across roles.	Online or offline per posture decision (stretch Director split keeps it online; ADR-0002 §4.3 step-5).	Verify signed by a threshold of snapshot keys; enforce version matches the one timestamp named; enforce not-expired; enforce targets version it lists is $\geq$ the device's last-seen.	Posture (online vs offline) UNVERIFIED pending the Director/Image split decision (ADR-0002 §4.3 step-5).
targets	Authorization + integrity binding: the map of target path → {length, hashes (SHA-256/512), custom metadata}. Names exactly which artifacts are authorized and their expected digests.	Offline (Image repo) and/or online per-device (Director repo, stretch §5).	Verify signed by a threshold of targets keys; enforce version matches snapshot; enforce not-expired; the requested artifact's path MUST be present and its length+SHA-256 MUST equal what the MVP gate independently	Delegations (hash-bin / succinct TAP-15) deferred; per-device Director targets is the §5 stretch, UNVERIFIED .

role	signs / asserts	key posture	device check (per refresh)	UNVERIFIED
			recomputes (§6, §8).	

The device-side guarantees the role chain produces: **freshness** (timestamp+expiry), **role consistency** (snapshot pins targets version/hash), **monotonicity** (every role's version strictly increases), **authorization** (the artifact appears in targets), and **digest binding** (targets length+SHA-256 == the bytes the device hashes). The artifact's existing ED25519 detached signature (MVP) is retained and re-checked unchanged — TUF is additive (ADR-0002 §3.2; §8 below).

§4. attack classes tuf adds over plain signing

Per ADR-0002 §3.1 / §4 and `research/stacks/tuf-go-tuf.md` §4, the device-side role enforcement closes attack classes that MVP plain ED25519+SHA-256+AVB leaves open (`security/signing_verification.md` §2, §11 item 4). The native A/B auto-rollback (§8 gate 4) remains the independent anti-bricking backstop regardless.

attack class	MVP plain-signing exposure	TUF role(s) that close it (device-side)
Arbitrary install / wrong-software	Closed only if the device already knows the correct hash and the key is intact.	targets (artifact must be a named, digest-matched authorized target).
Rollback (install an older, validly-signed-but-superseded build)	OPEN — old signed artifacts stay valid (<code>security/signing_verification.md</code> §11 item 4).	snapshot+targets monotonic version; device rejects a metadata version $\leq$ last-seen. Composes with AVB rollback-index anti-downgrade (<code>rollout_engine.md</code> / <code>research/stacks/android-avb-rollback.md</code>).
Indefinite freeze (pin a device to a stale-but-valid state)	OPEN — no freshness anchor.	timestamp short expiry; an expired/withheld timestamp is rejected, so a stuck repo is detectable, not silently accepted.
Fast-forward (jump version counter to lock out future updates)	OPEN.	TUF version-recovery semantics + root rotation re-baseline; device enforces ordered role recovery.
Mix-and-match (combine artifacts/	OPEN.	snapshot binds the consistent set of

attack class	MVP plain-signing exposure	TUF role(s) that close it (device-side)
metadata from different releases)		targets versions/ hashes; device rejects an inconsistent combination.
Endless-data / extraneous-dependencies	OPEN.	Metadata declares expected lengths; device caps reads at the targets-declared length.
Malicious-mirror denial	OPEN — a hostile mirror can serve stale/blocked content.	Threshold-signed, expiring metadata makes a mirror unable to forge currency; device detects stale/forged metadata.
Key-compromise recovery	OPEN — one key = full break.	Thresholds + offline root/targets keys; root rotation (§7) re-establishes trust after an online-key compromise without re-flashing devices.

These are the ADR-0002 §3.1 / §5.1 “*real attack classes in 1.0.1+*”.

UNVERIFIED: the exact recovery/version-rollback-protection semantics depend on the go-tuf/v2 client flow (updater/trustedmetadata), whose API is carried UNVERIFIED (ADR-0002 §8 item 2; §11 item 2).

§5. server-side device-side split

ADR-0002 splits the work along a hard seam: **the server library is decided and idiomatic; the device client is the dominant, OPEN cost** (ADR-0002 §3.2, §8 item 1).

Server-side — DECIDED (ADR-0002 §3.2, §4): publish + sign metadata with `github.com/theupdateframework/go-tuf/v2` (Apache-2.0; verified latest v2.4.2 / spec 1.0.31 as of 2026-06-07 per ADR-0002 §3.2). The publish path treats each artifact as an opaque target (`TargetFiles.FromFile(...)` → path + length + SHA-256/512), bumps snapshot + timestamp, signs, and writes metadata next to the byte-identical blobs, served over the existing HTTP/3→HTTP/2 + Brotli path (ADR-0002 §3.2). Server effort estimate LOW–MEDIUM, **UNVERIFIED until spiked** (ADR-0002 §8 item 7). ADR-0002 §4.3 step-1 prototypes this publish path **plus a Go reference refresh client** end-to-end to validate the model before the device client is even sized.

Device-side — OPEN, gated behind a spike (ADR-0002 §4.3 step-2, §8 item 1). No verified production-grade Android/Kotlin/JVM TUF client is known to exist (ADR-0002 §8 item 1; landscape pins `tuf-go-tuf` Android-15 fit at 2/5 on this gap). The on-device client path is one of two options the ADR-0002 spike **MUST** decide:

option	what it is	pros (per ADR-0002)	cons / risks (per ADR-0002)	status
A — gomobile-wrapped go-tuf/v2	Compile go-tuf/v2's metadata/updater + metadata/trustedmetadata client flow to an Android library via gomobile/JNI; the Kotlin agent calls the wrapped Go.	Reuses the same, audited go-tuf/v2 verification logic as the server; one trust implementation, no second hand-written verifier to keep correct.	Adds the Go toolchain to the Android build + an APK binary-size cost; gomobile/JNI boundary + Android-15/RK3588 integration are UNVERIFIED (ADR-0002 §5.2, §8 item 8). Means owning security-sensitive verification code (the highest-risk thing to hand-write); must re-derive TUF correctness independently; Security-KMP fit for TUF role/threshold/expiry primitives is UNVERIFIED (ADR-0002 §5.2, §8 item 9).	UNVERIFIED — spike must measure size JNI viability.
B — hand-rolled Kotlin TUF client	Implement the root→timestamp→snapshot→targets refresh/verify flow natively in Kotlin against Security-KMP crypto primitives.	No Go toolchain / no JNI / smaller footprint; fits the Kotlin agent natively.	Must re-derive TUF correctness independently; Security-KMP fit for TUF role/threshold/expiry primitives is UNVERIFIED (ADR-0002 §5.2, §8 item 9).	UNVERIFIED — spike must prove correctness + brick fit.

A third path — “*an existing JVM client*” — was considered in ADR-0002 §3.2 and found **none verified to exist or be maintained**, so it is not a recommendation here (ADR-0002 §8 item 1). Per HelixConstitution §11.4.6 (no-guessing) and §11.4.8 (research-before-implementation), this spec does **not** pick A or B: the choice is the ADR-0002 §4.3 step-2 spike's output, and device-side TUF enforcement is **not made mandatory** until that spike closes the UNVERIFIED items (ADR-0002 §4.3 step-4, §6).

Stretch — Director + Image split (ADR-0002 §4.3 step-5, §3.3 Option C verdict). A 1.0.1+ stretch hardening goal: a Go **Director** repo (online keys, mints **per-device** targets.json from the inventory DB) + an **Image** repo (offline-keyed). The per-device decision seam already exists — rollout_engine.md §4 deterministic cohort assignment is exactly the “what should this device install” decision a Director consumes (security/signing_verification.md §8 seam 4; ADR-0002 §4.2). The Primary/Secondary ECU tier is **vacuous** for full-power

Android devices and is **rejected**; each Helix device is a Primary doing full verification (ADR-0002 §3.3). aktualizr and full Uptane are **rejected** (ADR-0002 §3.3, §3.4). **UNVERIFIED**: no shipping production Android+Uptane integration is known (ADR-0002 §8 item 6).

§6. device-side verification flow (extends the verify-before-apply gate)

The MVP device gate (security/signing_verification.md §6) is, today: download byte-identical → recompute SHA-256/512 → verify ED25519 detached signature against the on-device trust-store key_id → hand the **fully verified local artifact** to ota-update-engine-bridge. The verify-gate is deliberately “*a distinct gate in front of the apply path so a TUF refresh/verify flow can be inserted ahead of the existing hash+signature check without changing the apply path*” (security/signing_verification.md §6, §8 seam 3; ADR-0002 §4.2).

1.0.1 inserts the TUF refresh/verify **in front of** that gate. Apply still requires a fully verified local file (verify-before-apply driver, ADR-0002 §2, §4.1) — TUF adds **authorization + freshness** ahead of the existing **authenticity + integrity** check; it does not remove either.

§6.1 flow (new TUF step prepended)

download byte-identical artifact (HTTPS+Range, no content-compression on the artifact path;

ADR-0004 §3.2/§3.3 – bytes the device hashes == bytes the server signed)



[NEW] TUF metadata refresh + verify — role order: root → timestamp → snapshot → targets

- refresh each role from the repo; verify threshold signatures per current `root`
- enforce expiry (reject expired timestamp/snapshot/targets/root)
- enforce monotonic `version` per role (reject $\leq$ last-seen → rollback/freeze guard, §4)
- enforce snapshot+targets consistency (mix-and-match guard, §4)
- resolve the requested artifact in `targets`; capture its authorized length + SHA-256
- ANY failure → ABORT before apply (record verdict; do NOT fall back to MVP-only)



MVP gate (unchanged, retained as defense-in-depth):

- recompute SHA-256 (and SHA-512 where available); MUST equal the manifest hash
- AND the TUF `targets` digest captured above (cross-binding, §8)
- verify ED25519 detached signature against on-device trust-store `key\_id`

▼
hand fully-verified local artifact to ota-update-engine-bridge → applyPayload

▼
update_engine FILE_HASH/METADATA_HASH + payload sig (native gate 3) → AVB / A/B (native gate 4)

§6.2 ordering rationale (consistent with the validation-chain-order audit)

The TUF step runs **before** the MVP hash/sig check, but the MVP **hash** still runs **before** the MVP **signature** (the verified S2-before-S3 invariant in security/validation_chain_order.md is preserved unchanged on the device side). The TUF targets digest and the device-recomputed SHA-256 MUST agree: TUF says “digest D is authorized”; S2 says “these bytes hash to D”; S3 says “D is signed by the trusted key”. Accept $\iff (TUF \text{ authorizes } D, \text{fresh} + \text{monotonic} + \text{consistent}) \wedge (\text{bytes hash to } D) \wedge (D \text{ signed by the trusted MVP key})$. A TUF-authorized digest that does not match the recomputed bytes is a payload-substitution attempt → ABORT (mirrors the S2-binds-D reasoning in validation_chain_order.md §why_order_matters).

§6.3 fields the spike must finalize (carried OPEN)

The concrete client API (metadata/updater, metadata/trustedmetadata) signatures, the on-disk trusted-metadata cache layout on Android, the refresh trigger relative to the update-check poll (rollout_engine.md §4 cohort check), and the exact failure→HTTP/agent-state mapping are **UNVERIFIED** and are the output of the ADR-0002 §4.3 step-2 spike (ADR-0002 §8 items 1–2; §11 below). This spec fixes the **flow and invariants**, not those signatures.

§7. key rotation + root-of-trust bootstrap on the device

§7.1 device root-of-trust bootstrap (trust-on-first-install)

TUF’s root is bootstrapped, not discovered: the device MUST start from a **pinned, trusted initial root.json** that did not come from the (untrusted) network. The bootstrap root is shipped **in the device build/image** — the same provenance as the MVP build-pipeline public key, which already “*lives in the device trust store*” and is “*embedded... non-empty... in the device build*” (security/signing_verification.md §4, §10 Layer 2). This keeps a single trust-anchor provenance: the verified-boot-protected image (AVB, native gate 4, §8) is the root of the root of trust. Subsequent root updates are accepted **only** if they chain-validate from this pinned root (§7.2). **UNVERIFIED:** the exact image location + AVB-coverage of the embedded root.json on Android-15/RK3588 is board-specific and carried UNVERIFIED (ADR-0002 §8 item 8).

§7.2 root rotation (chained root→root)

Root keys are **offline, threshold-held** (§3; ADR-0002 §2, §4.3 step-3). Rotation is a TUF chained walk: the device, holding trusted root version N , accepts root version $N+1$ iff $N+1$ is signed by a **threshold of version- N root keys** (and is itself well-formed, not-expired, version strictly increasing). The device walks the chain $N \rightarrow N+1 \rightarrow \dots \rightarrow \text{latest}$ before trusting any other role under the new root. This is what gives **key-compromise recovery** (§4): a compromised online key (timestamp/snapshot, or even a targets key) is revoked by publishing a new root, signed offline by the threshold, that drops the bad key — **without re-flashing devices**. A compromise of enough **offline root** keys to meet threshold is the catastrophic case requiring re-bootstrap (re-flash); thresholds make that the highest bar.

§7.3 rotation overlap with the MVP key_id trust store

The MVP already supports rotation overlap via key_id (each trust-store entry binds key_id $\rightarrow$ public key + algorithm; security/signing_verification.md §3, §4). TUF role-key rotation (§7.2) and MVP artifact-signing-key_id rotation are **independent** rotation tracks that coexist: TUF rotates *role* keys via root; the MVP rotates the *artifact* signing key via key_id in targets/trust-store. The device must tolerate both overlapping (accept old+new for a window) so a rotation does not strand in-flight devices (key_management.md, referenced; ceremony mechanics out of scope here per §1). **UNVERIFIED:** whether Security-KMP exposes the threshold/multi-key primitives a hand-rolled client (§5 option B) would need (ADR-0002 §8 item 9).

§8. composition with the ota-artifact-validator pipeline + native gates

§8.1 server side — TUF wraps, does not reorder, the S-stage pipeline

The server ota-artifact-validator pipeline is the verified, fail-fast **S2(hash)→S3(sig)→S4(version)→S5(target)→S6(metadata)** order (validation_chain_order.md, FACT). TUF server-side publish (§5) is **additive after validation**: an artifact is validated by the existing pipeline (unchanged), and only a passing artifact becomes a TUF **target** (path+length+SHA-256 $\rightarrow$ TargetFiles.FromFile) and gets metadata published. TUF does **not** reorder or replace S2→S3; the artifact stays byte-identical (ADR-0002 §3.2; security/signing_verification.md §8 seam 1). The server S3 verification key is, and remains, taken **only** from server config (resolvePublicKey), never from the request (validation_chain_order.md §http_mapping trust-boundary note) — TUF metadata does not change that boundary.

§8.2 device side — TUF is a NEW gate in front of the four existing gates

Adding the device TUF refresh/verify (§6) makes the device-side trust chain a **five-gate** chain. The four MVP/native gates (security/signing_verification.md §2 mermaid, §7) are unchanged:

gate	owner	what it adds	new in 1.0.1?
TUF metadata refresh/verify	Helix device agent (this spec)	authorization + freshness + monotonicity + role consistency (§3, §4)	YES (new) — runs first
Helix detached-sig + SHA-256 (device re-verify)	Helix agent (security/signing_verification.md §6)	authenticity + integrity, before apply	no (MVP, retained)
update_engine FILE_HASH/ METADATA_HASH + payload sig	AOSP	payload integrity during apply to inactive slot	no (native)
AVB / dm-verity	AOSP	verified boot of the running system	no (native)
A/B boot_control + auto-rollback	AOSP	atomic apply + automatic boot-failure rollback (zero-bricking, independent of any metadata layer)	no (native)

Byte-identity is the contract that lets the TUF gate coexist with all four: re-compressing/re-wrapping would break both the update_engine hash check and the TUF target entry (security/signing_verification.md §7; ADR-0004 §3.2). A failure at the new TUF gate aborts **before** update_engine is invoked (same discipline as the MVP gates); a boot-stage failure still triggers native A/B rollback regardless of TUF (ADR-0002 §5.1) — TUF never weakens the anti-bricking backstop, which is **why a safe MVP could ship without the device TUF client** (ADR-0002 §5.1, §8 item 1).

§9. data-model deltas (tuf metadata storage)

rollout_engine.md §11 explicitly **deferred** TUF metadata tables out of the rollout-engine spec to this document: “*This engine spec does not add TUF metadata tables or fields.*” The phase-1.0.1 README §6 lists “*plus TUF metadata storage*” among the migration-002 data-model deltas. The conceptual storage this spec requires (DDL owned by the migration spec, **not** fixed here):

- **TUF metadata blobs** — the signed root / timestamp / snapshot / targets JSON the server publishes and the device refreshes (server stores authoritative copies; served byte-identically over the existing path). Stored alongside artifact blobs in the Storage brick and/or a metadata table; consistent-snapshot versioning means root/targets are version-addressed.
- **Per-device trusted-metadata cursor** (forward / Director seam) — the last-seen role versions per device, so the device-side monotonicity check (§4 rollback/

freeze) and a future Director's per-device targets (§5 stretch) compose with the rollout_engine.md inventory/cohort model.

- **Root-rotation history** — the chained root versions for audit (§7.2).

UNVERIFIED / OPEN: exact tables, columns, and whether metadata lives in Postgres vs the Storage object store are the data-model owner's decision (successor to migration_002_design.md); this spec states the *requirement*, not the schema. Consistent-snapshot vs non-consistent storage layout depends on the go-tuf/v2 publish config (ADR-0002 §3.2), carried UNVERIFIED until the ADR-0002 §4.3 step-1 server prototype.

§10. testing (four-layer + attack-class)

Per HelixConstitution §1 (UNVERIFIED clause) and 1.0.1-staged-rollout/README.md §8, device TUF is a **safety-critical path targeting ≥90% coverage** and ships all four layers with no-bluff positive evidence (§7.1), plus the attack-class suite the README §8 mandates.

- **Layer 1 — source-presence gate.** Static check that the device path actually contains: a TUF refresh/verify step that runs **before** the MVP hash/sig gate (§6); role-order enforcement root→timestamp→snapshot→targets; expiry checks; per-role monotonic-version checks; snapshot[?]targets consistency; the TUF-digest [?] recomputed-SHA-256 cross-binding (§6.2); the pinned-bootstrap-root presence (§7.1). Absence of any is a build-time failure.
- **Layer 2 — artifact gate (bytes/metadata shipped).** Confirm artifacts stay **byte-identical** (no re-wrap), the device image embeds a non-empty **pinned root.json** plus the MVP trust-store, and published metadata (root/timestamp/snapshot/targets) is well-formed + threshold-signed for the released set.
- **Layer 3 — runtime / attack-class (must reject, not apply).** End-to-end: publish metadata → device refresh+verify → MVP gate → apply. **Negative attack-class cases (README §8): rollback** (offer an older monotonic metadata version → reject); **freeze** (expired/withheld timestamp → reject); **mix-and-match** (inconsistent snapshot[?]targets set → reject); malicious-mirror stale metadata → reject; TUF-authorized digest ≠ recomputed bytes → reject; threshold not met / signed by a rotated-out role key → reject; expired root/targets → reject. Confirm AVB/A/B still auto-rolls-back on a corrupt-slot boot **independent** of the TUF gate (ADR-0002 §5.1).
- **Layer 4 — mutation meta-test (PASS→FAIL on negation).** Mutate and require the suite to flip PASS→FAIL: skip the TUF refresh entirely (fall straight to the MVP gate); force role-signature verify to always-true; accept a non-monotonic role version; skip the expiry check; accept an inconsistent snapshot; drop the TUF-digest[?]SHA-256 cross-binding; trust an unchained root. A surviving mutant is a coverage defect on a safety-critical path.

Real-device validation on Orange Pi 5 Max (README §8): staged cohort, induced regression → automatic halt, **authorized** rollback, with the device TUF gate active — once the §5 client is chosen and the spike closes the OPEN items (not before; ADR-0002 §4.3 step-4).

§11. open / unverified items

Carried forward (must close before **device-side TUF enforcement is made mandatory** — ADR-0002 §4.3 step-4, §6 Status):

1. **On-device TUF client path (gomobile/JNI vs hand-rolled Kotlin)** — dominant adoption cost; no verified production Kotlin/JVM TUF client is known. The §5 A-vs-B choice is the ADR-0002 §4.3 step-2 spike's output, not decided here. **UNVERIFIED/OPEN**. (ADR-0002 §8 item 1.)
2. **go-tuf/v2 client-flow API** (metadata/updater, metadata/trustedmetadata) signatures not fully enumerated — client sizing + §6.3 fields provisional until confirmed against pkg.go.dev. **UNVERIFIED**. (ADR-0002 §8 item 2.)
3. **Security-KMP (device) / security (server) fit for TUF role/threshold/offline-key/expiry primitives** — whether the catalogue bricks expose or can host these without bespoke crypto is not confirmed; bears directly on §5 option B feasibility and §7.3. **UNVERIFIED**. (ADR-0002 §8 item 9; security/signing_verification.md §11 item 2.)
4. **Android-15 / RK3588 board specifics** the device client integrates against (update_engine constants/AIDL; the AVB-protected on-image location of the embedded root.json, §7.1). **UNVERIFIED**. (ADR-0002 §8 item 8; security/signing_verification.md §11 item 3.)
5. **Snapshot key posture** (online vs offline) and **Director/Image split** adoption — the §5 stretch; no shipping production Android+Uptane integration is known. **UNVERIFIED**. (ADR-0002 §8 item 6; §3.3.)
6. **TUF metadata storage DDL** (§9) — tables/columns and Postgres-vs-object-store placement are the data-model owner's decision (migration successor), not fixed here. **OPEN**.
7. **Adoption-effort figures** (server LOW–MEDIUM; key-mgmt MEDIUM–HIGH; device HIGH) are estimates, not measured. **UNVERIFIED** until spiked. (ADR-0002 §8 item 7.)
8. **HelixConstitution clause numbers** (§11.4.61, §7.1, §11.4.6, §11.4.8, §11.4.28, §11.4.74, §11.4.123, §11.4.125, §1) are **UNVERIFIED** against the authoritative constitution text (corpus convention; the source file is not present in this repository — ADR-0002 §7 note).

§12. compliance notes (helixconstitution)

clause (UNVERIFIED numbers)	how this spec complies
§11.4.61 (ToC)	ToC present immediately after the metadata table. Every claim cites ADR-0002, the MVP signing spec, the validation-chain-order audit, or the rollout-engine spec; the device client (A vs B), library client
§7.1 / §11.4.6 (no-bluff / no-guessing)	API, catalogue-brick fit, board specifics, snapshot posture, and storage DDL are carried as UNVERIFIED/OPEN rather than asserted (§11). No A/B client recommendation is invented.

clause (UNVERIFIED numbers)	how this spec complies
§11.4.8 (research-before-implementation)	Device-side TUF enforcement is gated behind the ADR-0002 §4.3 step-2 spike; §5/§6.3 explicitly leave the client and its API as spike outputs; nothing on-device is mandated without that research evidence.
§11.4.28 (decoupling)	The TUF gate is a distinct device step in front of the unchanged MVP verify-gate and the unchanged native gates (§6, §8); server TUF publish is additive after the unchanged ota-artifact-validator pipeline — each independently testable and Uptane-swappable.
§11.4.74 (catalogue-first reuse)	Server publish via go-tuf/v2; device crypto routed through Security-KMP; no bespoke crypto recommended where a brick may exist (the brick fit is the UNVERIFIED §11 item 3, not an invented dependency).
§1 / §1.1 (four-layer + mutation)	§10 specifies all four layers + the attack-class suite (rollback/freeze/mix-and-match) on the $\geq 90\%$ safety-critical device-trust path.
§11.4.123 (rock-solid proof)	UNVERIFIED items are closed by the ADR-0002 §4.3 spikes and Layer-3 attack-class runtime evidence, not by assertion; enforcement is not mandated until they close.
§11.4.125 (code-review gate)	Subject to the mandatory adversarial code-review subagent before acceptance (per the corpus convention).

§13. sources

All paths relative to docs/research/main_specs/:

- [research/adr/adr-0002-supply-chain-trust.md](#) — §1 (TUF is trust not delivery), §2 (drivers), §3.1–§3.4 (options: plain / TUF / Uptane / aktualizr; verified go-tuf/v2 facts; rejected items), §4.1–§4.3 (phased decision; MVP-forward seams; 1.0.1+ sequencing incl. step-2 client spike, step-3 ceremony, step-5 Director split), §5 (consequences), §6 (status/gating), §8 (carried UNVERIFIED items 1–9).
- [1.0.0-mvp/security/signing_verification.md](#) — §2 (MVP trust model + accepted residual), §3 (ED25519 default + integrity primitives), §4 (build-pipeline key in device trust store), §6 (device re-verify-before-apply gate this spec extends), §7 (composition with AVB/A/B), §8 (TUF-forward seams: opaque target, signer abstraction, verify-gate-distinct, per-device decision), §10 (four-layer testing), §11 (UNVERIFIED items).

- 1.0.0-mvp/security/validation_chain_order.md — verified S2(hash)-before-S3(signature) order, signature-binds-to-the-hashed-bytes, the resolvePublicKey server trust-boundary, why-order-matters.
- 1.0.1-staged-rollout/README.md — §4 (device-side TUF goal), §6 (TUF metadata storage data-model delta), §7 (device-side TUF in ota-android-agent), §8 (attack-class testing + real-device validation).
- 1.0.1-staged-rollout/rollout_engine.md — §4 (deterministic cohort = per-device decision seam a Director consumes), §11 (TUF metadata storage deferred to this document).
- 1.0.0-mvp/security/key_management.md — key custody/rotation mechanics (referenced; ceremony out of scope here).
- ADR-0002 evidence base: research/stacks/tuf-go-tuf.md (§3 roles, §4 attack classes, §5 verified library facts, §7 effort, §9 recommendation); research/stacks/uptane.md (§3 dual-repo, §6 ECU model rejected, §9–§10 relevance/recommendation) — cited transitively via ADR-0002.